

Abundance Federation

Data Pipeline Security & Validation

A Phased Roadmap From Pilot to Public

Building toward 2027 public submissions, starting with what matters today

April 2026

Why This Document Exists

Right now we'll be working with a small group of trusted volunteers — schools we've onboarded ourselves, charity partners, and adult volunteers who come in through warm introductions. The threat surface is small. We could probably get away with a fairly minimal validation layer.

By 2027 we're planning to open dataset creation to the wider public. At that point everything changes. We'll have anonymous submitters at scale, automated abuse attempts, the possibility of malicious uploads, and reputational risk if something inappropriate makes it into a published dataset. Building for that future starts now.

This document is the team's reference for what we build, when we build it, and why each piece matters. It's organised into four priority tiers:

- **P0 (build now):** Essential during the current sprint. Cheap, foundational, and prevents real problems even with trusted volunteers.
- **P1 (build through 2026):** Required before we open to public submissions. Most can be designed now and built progressively, with feature flags so they can be enabled when ready.
- **P2 (build when capacity allows):** Worth doing eventually, but team time is better spent elsewhere first.
- **P3 (defer):** Not bad ideas, but they solve problems we don't have and may never have. Skip unless specific signals emerge.

Each item below explains what the threat is, what happens if we don't address it, and how we build the protection. Understanding the "why" matters — it lets the team make sensible judgement calls when implementation details come up that aren't covered here.

P0: Build Now

These twelve items are essential for the current sprint or the period immediately after. They prevent crashes, protect volunteer privacy, and lay the foundations that everything else depends on. They're cheap to implement and the cost of skipping them is high even with trusted volunteers.

P0 — 1. File size hard cap (pre-parse)

The threat: A volunteer—accidentally or otherwise—uploads a very large file (e.g., a 5GB video instead of a photo). The ingestion pipeline opens it with Pillow before any size check, exhausting server memory.

If we skip this: The server runs out of RAM and crashes. Other volunteers' submissions fail silently. The team spends time debugging an issue that has nothing to do with the actual data. If this happens during a school field trip, an entire class's submissions could be lost.

How we build it: Before opening any file with an image library, check the byte size on disk. Reject anything over a hard ceiling (25MB is generous for a phone photo and tight enough to prevent abuse). Log the rejection with the volunteer ID and original filename so we can follow up on legitimate cases.

P0 — 2. Image corruption detection

The threat: A volunteer's upload is interrupted (poor mobile signal, app crash, or a corrupted file from a faulty phone). The file looks like an image but can't actually be opened.

If we skip this: The ingestion script crashes when it tries to process the file. Without graceful handling, this can halt the entire batch — every image submitted that day might be stuck in limbo. Volunteers wonder why their work disappeared. Trust erodes.

How we build it: Use Pillow to attempt to open and verify each image before further processing. If verification fails, move the file to a quarantine folder and log the issue with the volunteer ID. The volunteer can be asked to resubmit. The pipeline continues with the rest of the batch.

P0 — 3. Format whitelist

The threat: Someone uploads a file that looks like an image but is actually something else — an executable disguised with a .jpg extension, a malformed file designed to exploit image library vulnerabilities, or a format the pipeline can't safely handle.

If we skip this: Image libraries like Pillow have had serious vulnerabilities in the past, often in handling of unusual formats (TIFF, GIF, BMP). Even when patched, accepting a wide range of formats means we're trusting upstream code more than necessary. A maliciously crafted file could potentially execute code on the server.

How we build it: Accept only JPEG, PNG, and optionally WebP. Validate by reading the first few bytes (the file's "magic number"), not just the extension — file extensions can be spoofed. Convert everything to clean JPEG in sRGB colour space as a normalisation step. This gives us one consistent format downstream and strips away any embedded surprises.

P0 — 4. Image bomb protection

The threat: A small file (a few hundred kilobytes) decompresses to an enormous image when opened — say, 50,000 × 50,000 pixels. This is called a "decompression bomb" and it's a known attack pattern against image processing systems.

If we skip this: The pipeline tries to load the image into memory. The 200KB file becomes 7.5GB in RAM. The server runs out of memory and crashes, or worse, swaps to disk and

becomes unresponsive for a long time. This is a denial-of-service attack with almost no effort from the attacker.

How we build it: Set Pillow's `Image.MAX_IMAGE_PIXELS` to a reasonable limit (e.g., 100 million pixels, which still allows a 10000×10000 image — far more than any phone camera produces). Pillow will warn or refuse to open files that exceed this. Combine with the file size cap (item 1) for layered protection.

P0 — 5. EXIF metadata stripping

The threat: Phone photos contain EXIF metadata: GPS coordinates of where the photo was taken, the device model, the photographer's name in some cases, sometimes even network identifiers. Some of our volunteers are minors. Some photos are taken in their gardens or near their schools.

If we skip this: If we publish datasets without stripping EXIF, we're publishing the home or school locations of children. That's a serious safeguarding failure with potential legal and reputational consequences. Even if we don't publish images directly, the EXIF data sits in our storage indefinitely — a data protection liability.

How we build it: Strip all EXIF metadata on ingestion. The only field worth keeping is image orientation (otherwise photos may display rotated). Use Pillow's built-in capabilities or a dedicated library like `piexif`. This is a one-line change in the right place and protects every volunteer forever.

P0 — 6. Filename sanitisation

The threat: When we rename files using volunteer-submitted metadata (volunteer ID, farm name, capture date), those inputs flow into shell commands and filesystem paths. A malicious or accidental input like `../../etc/passwd` or `farm; rm -rf /` could cause path traversal or shell injection.

If we skip this: Path traversal could let an attacker overwrite system files or read sensitive data. Shell injection is even worse — it could allow arbitrary code execution on the server. Even non-malicious inputs (a farm name with a slash, or with quotes) can break the pipeline silently and corrupt the renaming logic.

How we build it: Validate every metadata field used in filenames against a strict allowlist: alphanumerics, underscores, and hyphens only. Reject anything else with a clear error. Never construct shell commands by concatenating user input — use library functions that handle escaping properly. This is one of the most important security controls on the list.

P0 — 7. Exact duplicate detection

The threat: Volunteers may accidentally submit the same photo twice — the app glitches, they're not sure if their first submission worked, or they hit upload twice. Without detection, the same image enters the dataset multiple times.

If we skip this: Duplicate images skew the dataset. They make some classes appear more frequent than they really are, biasing any model trained on the data. They also make accuracy metrics meaningless — if the same image appears in training and test sets, the model looks better than it actually is. At scale, accidental duplicates can be a significant fraction of submissions.

How we build it: Compute a SHA-256 hash of each incoming file. Maintain a running record of all hashes (in a CSV or small SQLite database). New submissions matching an existing hash are flagged as duplicates and excluded from the active dataset. Trivial to implement, runs in milliseconds per file, prevents a real and common problem.

P0 — 8. CSV parsing robustness

The threat: KoboToolbox CSV exports may contain unusual characters: curly quotes from copy-pasted text, em-dashes, accented characters, BOM markers, or unexpected line endings (especially from Windows users).

If we skip this: A naïve CSV parser breaks on these. The pipeline fails on perfectly innocent submissions. The team spends time figuring out why “nothing is working” when the actual issue is one volunteer who wrote a note containing a smart quote.

How we build it: Use Python’s built-in `csv` module with UTF-8-sig encoding (handles BOM markers automatically). Wrap parsing in try/except blocks that log problematic rows but continue processing the rest of the file. Never let one bad row kill an entire batch.

P0 — 9. Text sanitisation

The threat: Free-text fields (notes, custom farm names, descriptions) may contain anything: HTML tags, control characters, very long strings, content that breaks downstream tools.

If we skip this: Without sanitisation, a stray HTML tag could cause display issues in dashboards. Control characters could break log files. Excessively long strings could break the database. Worse, if any of this text is later displayed in a web interface without escaping, it becomes a cross-site scripting vector.

How we build it: Strip control characters, normalise whitespace, enforce maximum lengths (e.g., 500 characters for notes), and HTML-escape on output. Don’t try to clever-clean the input — just store it safely and escape it when displaying. Standard practice but easy to forget.

P0 — 10. Audit logging

The threat: Without logs, we have no visibility into what the pipeline is doing. We can’t tell whether issues are happening, we can’t investigate volunteer complaints (“my submission isn’t showing up”), and we can’t demonstrate compliance with data protection requirements.

If we skip this: Problems go undetected until they’re severe. The first time we hear about an issue is when it’s already affected many volunteers. We can’t answer basic questions like “how many submissions did we get last week?” or “did this volunteer’s file actually arrive?” Investigating anything becomes archaeology.

How we build it: Every ingestion run produces a structured log: timestamp, volunteer ID, original filename, file size, validation results (passed/failed each check, with reasons), final disposition (accepted, quarantined, rejected). Store logs in a clear, searchable format. Don’t log image content — just metadata. Foundation for everything else, including the QA framework.

P0 — 11. Volunteer ID validation

The threat: A submission arrives with a volunteer ID that doesn’t match anyone in our active volunteer list. Most likely a typo, but possibly someone using a guessed or invented ID.

If we skip this: Without validation, the submission gets processed and attributed to a non-existent volunteer. That image is now in the dataset but disconnected from any real contributor — we can’t track quality, can’t give credit, can’t investigate if there’s an issue. At scale, this becomes a meaningful pollutant in the data.

How we build it: Maintain a list of active volunteer IDs (already needed for the QA framework). On ingestion, check the submitted ID against this list. Unknown IDs go to a quarantine folder for the coordinator to review — it’s usually a typo that can be fixed manually, occasionally something to investigate further.

P0 — 12. Manual safeguarding review during pilot

The threat: We're working with minors. Even from trusted school volunteers, there's always a small chance of inappropriate content being submitted — a wrong file selected, a prank, or genuine harmful intent.

If we skip this: If inappropriate content reaches Label Studio where other volunteers (including students) can see it, the consequences are severe: safeguarding failure, possible legal exposure, immediate damage to trust with schools, and a story that could end the project. Even one incident is unacceptable.

How we build it: During the pilot, the coordinator reviews every batch of submissions before they're imported into Label Studio. This isn't scalable to thousands of volunteers, but it's the right control during the pilot phase. P1 introduces automated detection for when manual review can't keep up.

P1: Build Through 2026

These eight items shift our posture from “trust but verify” to “verify because we can’t trust.” They’re required before we open dataset creation to public submissions in 2027. Most can be designed and partially built during 2026, with feature flags allowing them to be enabled when public submissions begin.

P1 — 13. GPS bounds validation

The threat: Public submitters might upload random photos from anywhere — holiday snaps, photos found online, deliberately misplaced submissions to game any geographic incentives in our data collection.

If we skip this: If we’re building an Irish weed dataset and accept photos from Spain, the dataset becomes useless for its purpose. Models trained on it would be confused by the irrelevant data. The project’s claim to produce domain-specific Irish datasets becomes harder to defend.

How we build it: Validate GPS coordinates against the bounding box of the relevant geography (Ireland: 51.5–55.5° N, -10.5 to -5.5° W). Submissions outside the box are flagged for review rather than auto-rejected — there are legitimate edge cases (offshore islands, GPS drift) that warrant human judgement. With trusted volunteers this is a nice-to-have; with public submitters it’s essential.

P1 — 14. Image dimension limits

The threat: Public submitters may upload very small images (irrelevant to ML training) or very large images (storage and processing costs). Without limits, the dataset accumulates noise on both ends.

If we skip this: Below ~100px on any side, an image is too low-resolution to be useful for almost any computer vision task. Above ~4000px, the diminishing returns hit hard — we’re storing four times the data for negligible information gain. At public scale, both ends waste storage and processing time.

How we build it: Reject images below 100px on any side outright. For images above 4000px, downsample to 4000px on the longest edge before storage. This preserves quality where it matters and trims excess where it doesn’t. Implement during 2026 when storage costs become more visible.

P1 — 15. Timestamp sanity

The threat: Submissions arrive with timestamps that don’t make sense: dates in the future (sometimes due to misconfigured device clocks, sometimes deliberate), or dates from years ago (recycled photos rather than freshly captured).

If we skip this: Future dates corrupt time-based analysis (you can’t plot weekly submissions if some fall in next year). Very old submissions undermine the dataset’s claim to be current. Both are signs that should trigger review rather than silent acceptance.

How we build it: Reject or flag submissions with timestamps more than 1 day in the future or more than 1 year in the past. “Flag” rather than “reject” for the past case — there are legitimate uses (annotated archives, retrospective contributions) but they should be deliberate, not accidental.

P1 — 16. Capture condition consistency

The threat: A submission claims it was taken in “morning” and “night” simultaneously, or “sunny” and “wet” in conditions that contradict the GPS-derived weather data for that location at that time.

If we skip this: Inconsistent metadata is a signal of careless submission, automated abuse, or someone gaming the system to look like they’ve contributed in more diverse conditions than they actually have. At scale, this corrupts diversity metrics that downstream researchers may rely on.

How we build it: Define a small set of mutually exclusive conditions (morning vs. night vs. afternoon vs. evening). Reject submissions that select multiple incompatible options. For weather conditions, cross-reference against historical weather data for the location/time when available. Build during 2026.

P1 — 17. Categorical normalisation

The threat: Public submitters type farm names freely: “Smith Farm,” “smith’s farm,” “Smith Farms Ltd,” “THE SMITH FARM” — all the same place but stored as four different categories.

If we skip this: Without normalisation, the dataset shows four farms instead of one. Per-farm analysis becomes meaningless. Reports to partners look unprofessional. The QA framework can’t track per-location quality properly. At scale, this is a serious data quality issue.

How we build it: Maintain canonical lists for known categorical fields (farm names, capture conditions, regions). Implement fuzzy matching on submission to suggest the canonical version. For new entries, route through a moderation queue rather than accepting them automatically. Becomes essential when the volunteer pool moves beyond people we can directly coordinate with.

P1 — 18. Automated NSFW detection

The threat: At public scale, we cannot manually review every submission. Inappropriate content — sexual imagery, graphic violence, hate symbols — may be uploaded deliberately or accidentally.

If we skip this: Reputational damage from inappropriate content reaching a published dataset would be severe. If a TY student opens Label Studio and sees something inappropriate that another submitter uploaded, the safeguarding failure is immediate and serious. Schools would (rightly) leave the platform. Media coverage would be devastating. This is the single highest-priority item in P1.

How we build it: Open-source NSFW detection models (e.g., NudeNet, Yahoo’s open_nsfw) can flag inappropriate content automatically with high accuracy. Flagged submissions go to a moderation queue for human review rather than auto-rejection — false positives are real (medical images, art photography, breastfeeding photos all sometimes flagged). Build this with feature flags so we can enable it the moment public submissions begin.

P1 — 19. Rate limiting per volunteer

The threat: A public submitter could submit hundreds or thousands of low-quality images to game any leaderboard, points system, or visibility metric. Or an automated abuse system could flood the pipeline.

If we skip this: Without rate limits, gaming the system is trivial. The volunteer of the month is whoever runs a script. Genuine high-quality contributors get drowned out. The QA framework can’t cope with the volume of low-quality submissions to filter through. Server costs spike unexpectedly.

How we build it: Cap submissions at a reasonable daily ceiling per account (e.g., 100/day). For verified power users, allow higher ceilings on request. Implement at the application layer,

not the network layer — we want to limit per volunteer account, not per IP (multiple legitimate users may share an IP, like a school).

P1 — 20. Submission session validation

The threat: Without session validation, anyone can hit the submission API directly without going through the official app or web interface. This bypasses any client-side validation, rate limiting, or user experience design.

If we skip this: Direct API submissions are how automated abuse happens at scale. A bad actor can write a script that submits thousands of garbage images in minutes. Even well-meaning developers might integrate against the API in unexpected ways that cause issues. The pipeline should accept submissions only through official channels.

How we build it: Submissions must include a valid session token issued by the official app or web interface. Tokens expire after a short period and are tied to authenticated user sessions. The submission API rejects requests without valid tokens. Standard pattern in any production system; essential before public launch.

P2: Build When Capacity Allows

These four items are worth doing eventually but the team's time is better spent on P0 and P1 first. Build when there's capacity, or when specific signals suggest they're needed.

P2 — 21. Near-duplicate detection

The threat: Volunteers (or scripts) submit slightly modified versions of the same image — cropped differently, lightly filtered, or rotated — to inflate submission counts or game diversity metrics.

If we skip this: More about dataset quality than security. At scale, near-duplicates are a significant pollutant: they make the dataset less diverse than it appears, can leak between train/test splits, and can be used to game any contribution-counting system. But the risk is gradual rather than acute.

How we build it: Use perceptual hashing (pHash, dHash, or aHash) to compute a small fingerprint of each image. Images with Hamming distance below a threshold (e.g., 5) are flagged as near-duplicates. Implement after the basic duplicate detection is working and providing useful data on actual duplicate rates.

P2 — 22. Geographic impossibility detection

The threat: An account submits images from Dublin and then from Cork twenty minutes later. Either the GPS data is wrong, the account has been compromised, or someone is sharing credentials.

If we skip this: Useful signal for account compromise or credential sharing, but high implementation cost relative to the threat. We'd need to track submission sequences per volunteer, calculate distances and travel times, and decide on plausible thresholds. Worth building eventually but not urgent.

How we build it: Track the timestamp and GPS of each volunteer's recent submissions. Calculate implied travel speed between consecutive submissions. Flag anything implying movement faster than 200 km/h for review. Account for legitimate edge cases (group submissions during a school trip, where multiple students upload from one bus).

P2 — 23. IP-based geographic logging

The threat: Coordinated abuse — multiple accounts submitting suspiciously similar content from related IP addresses — is hard to detect without IP-level visibility.

If we skip this: We're not looking to block IPs, but logging them allows post-hoc analysis: "Were these 50 weird submissions all from the same IP range?" That kind of pattern detection is impossible without the data. The cost of logging is low; the cost of not logging only becomes visible after an incident.

How we build it: Log the IP address of each submission alongside the metadata we already capture. Don't use it for blocking decisions — use it only for retrospective analysis when something looks off. Be transparent with volunteers in the privacy policy about this logging.

P2 — 24. Burst-rate anomaly detection

The threat: Sudden spikes in submission volume could indicate viral success, coordinated abuse, or system misuse. Without monitoring, we don't know which.

If we skip this: Less heavy-handed than rate limiting; complements it. Where rate limiting prevents excessive use per account, burst detection identifies system-wide patterns that warrant attention. Useful for capacity planning as well as abuse detection.

How we build it: Monitor submission volume per hour and per day. Compare against a rolling baseline. Flag deviations beyond, say, 3 standard deviations for review. Don't auto-block — the spike might be a school class submitting together, or a media mention driving genuine engagement.

P3: Defer Indefinitely

These three items aren't bad ideas in principle, but they solve problems we don't actually have and may never have. Skip unless specific signals emerge that we need them.

P3 — 25. Irish IP blocking

The threat: The original suggestion was to verify submitter IPs against known Irish ranges, blocking or flagging non-Irish addresses.

If we skip this: Volunteers will be on school WiFi (which sometimes routes through unexpected places), home broadband (which may have non-Irish-looking IPs depending on the ISP), mobile networks (which often appear non-local), and perhaps VPNs they don't even know they have. The false positive rate would be enormous, generating constant noise. The GPS bounds check (P1, item 13) does this job better and more reliably.

How we build it: Don't implement. Use GPS validation instead. If, in future, we detect specific abuse patterns from particular geographic origins, we can implement targeted blocking based on actual evidence rather than blanket policy.

P3 — 26. Reputation scoring (separate system)

The threat: The original suggestion was a per-volunteer quality score that flags low performers based on submission patterns and validation failures.

If we skip this: This is conceptually similar to what Prof. Wilson is helping us design with the QA framework — statistical measurement of annotator and contributor reliability against gold-standard data. Building a parallel "reputation" system on top of validation failures would be less rigorous and would conflict with the framework. One source of truth on contributor quality is better than two.

How we build it: Don't build a separate reputation system. Invest the effort in the QA framework instead. The validation failures captured by the audit logs (P0, item 10) become input data to that framework rather than a separate scoring mechanism.

P3 — 27. Processing throttling at server level

The threat: The original suggestion was to throttle ingestion processing if spikes occur, or alert on sudden volume increases.

If we skip this: Our ingestion script runs as a scheduled job (e.g., nightly), not in response to incoming requests. There's no real attack surface here — abuse against the submission API is a separate concern handled by rate limiting and session validation. Throttling the ingestion job itself would only delay legitimate processing.

How we build it: Don't implement. The capacity concerns this would address are better handled by the burst-rate anomaly detection in P2 (item 24) at the submission layer, not the ingestion layer.

Build Order and Dependencies

A few items are foundational — they enable other items to work properly. Build them first within their tier:

- **Item 10 (Audit logging)** is foundational. Without it, we can't investigate problems, can't feed the QA framework, and can't demonstrate compliance. Build first.
- **Item 11 (Volunteer ID validation)** depends on having a list of active volunteers, which is also needed by the QA framework. Build alongside item 10.
- **Item 5 (EXIF stripping)** is essential before any processed image is moved into a folder accessible to other volunteers. Build before any cross-team integration testing involves real photos.
- **Item 18 (NSFW detection)** is the highest-priority P1 item. Public submissions cannot launch without it. Plan the build for late 2026, with feature flags ready before public launch.
- **Items 19 and 20 (rate limiting and session validation)** are tightly coupled. Build them together as a single "submission abuse prevention" workstream during 2026.

Most P0 items can be built in parallel by different team members. The dependencies above are about logical ordering for review and integration, not about strict sequencing.

Closing Thoughts

Two principles run through this whole document and are worth keeping in mind during implementation:

Logging before blocking. Where there's a choice between flagging suspicious activity for review and automatically blocking it, prefer flagging. Automated blocking creates false positives, frustrates legitimate users, and eliminates the audit trail we need to understand whether the blocking is even correct. We can always tighten controls later based on evidence; we can't recover trust lost from over-aggressive automation.

Feature flags for everything in P1. Every P1 item should be implemented behind a feature flag that can be enabled or disabled without code changes. This lets us turn on protections progressively as we move toward public launch, without big-bang deployments. It also gives us a safety valve if a control turns out to be too aggressive in production.

This document will evolve. As we learn what threats actually emerge and which controls are most effective, items will move between priority tiers. Please raise points of disagreement — the goal is to get to a position where the team is collectively confident that we're building the right things in the right order.